

CSIS 2300 Introduction to Algorithms

Spring 2026 – Homework 6 – **Solutions**

Instructions: You do not need to typeset your solutions; however, your submission must be a single PDF file. The assignment is due by **8:59pm on Tuesday, March 10**. Remember that you may submit as many times as you wish before the deadline (we'll only grade the last one), but late submissions will receive no credit – so submit early, submit often!

This assignment consists of problems numbered 1-6; be sure to look at the second page.

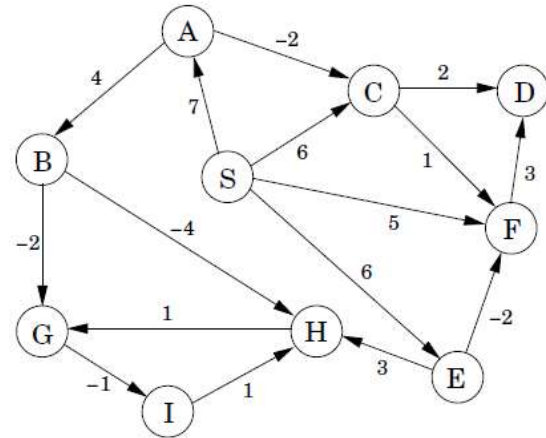
(1) A collection of n student workers at a campus center each work a single contiguous shift per week (e.g. Mondays 6-10pm, Wednesdays 10am-3pm); many of these shifts overlap. A supervisor wants to form a committee of these student workers to collect feedback from them; to ensure that the committee is representative, they insist that every one of the n workers must have a representative on the committee whose shift overlaps with their own. (One committee member can represent multiple workers.)

Design and state a linear-time greedy algorithm for solving this problem with the smallest possible committee that meets the representation requirement. Prove that your algorithm produces a valid answer and runs in $O(n \log n)$ time.

The solution here is a sort of “double greedy” approach. First, find the student whose shift ends earliest; call this time t_1 . Then find the student s whose shift starts before t_1 and ends latest; let the end time of s 's shift be t_2 . Add s to the committee and remove from consideration all students whose shifts start before t_2 . Repeat until there are no students left.

If we initially produce two lists of the students, one sorted by start time and one sorted by end time, then everything else is linear. Thus, the full process is $O(n \log n)$.

(2) Execute the Bellman-Ford algorithm on the graph shown at right. Your response should be a table similar to the one we created in class. Explain how your table verifies that the graph has no negative cycles.



The table has two consecutive columns with no change; this can only happen if there are no negative cycles. (Note this also tells that the lowest-weight path with the most links has five links, since the table repeats after column 5. That path is S -> A -> B -> H -> G -> I.)

Round->	0	1	2	3	4	5	6
S	0,--	0,--	0,--	0,--	0,--	0,--	0,--
A	∞	7,S	7,S	7,S	7,S	7,S	7,S
B	∞	∞	11,A	11,A	11,A	11,A	11,A
C	∞	6,S	5,A	5,A	5,A	5,A	5,A
D	∞	∞	8,F/C	7,F/C	7,F/C	7,F/C	7,F/C
E	∞	6,S	6,S	6,S	6,S	6,S	6,S
F	∞	5,S	4,E	4,E	4,E	4,E	4,E
G	∞	∞	∞	9,B	8,H	8,H	8,H
H	∞	∞	9,E	7,B	7,B	7,B	7,B
I	∞	∞	∞	∞	8,G	7,G	7,G

(3) Often there are multiple shortest paths between two nodes in a graph, especially when the graph is unweighted. Design and state an efficient algorithm for counting the number of shortest paths from vertex a to vertex z in a given undirected, unweighted graph G . State and justify the runtime of your algorithm.

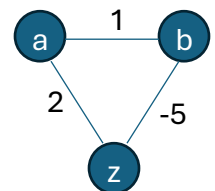
Modify breadth-first search by adding a count to each node; initialize the count for the source node to 1 and all other nodes to 0. Each time an edge is explored: (a) If it is a tree edge, add the parent node's count to the child node's count. (b) If it is a cross edge AND the nodes are on different levels (recall this means that the nodes are precisely one level apart), add the count for the node closer to the source to the count for the node further from the source. Once node z is added to the tree, we must make sure that we check if it has any cross edges before stopping.

Since BFS is $O(V + E)$ and we are adding a constant amount of work per edge, this algorithm will also be $O(V + E)$.

(4) Consider the following algorithm for finding the shortest path from node a to node z in a graph that contains some negative edges (but no negative cycles): Let $-w$ be the negative weight in the graph with the largest magnitude. Add $(w + 1)$ to the weight of every edge in the graph, so that all of the weights are positive. Now run Dijkstra's algorithm on the resulting graph.

Is this a valid algorithm? Either prove that it works correctly or give a counterexample.

It is not. Consider the graph at right (or any graph with a negative cycle). There is no shortest path from a to z since you can always follow the negative cycle one more time to get a lower weight. But Dijkstra's on the suggested modified graph will find one. (Length 8 in the pictured example.)



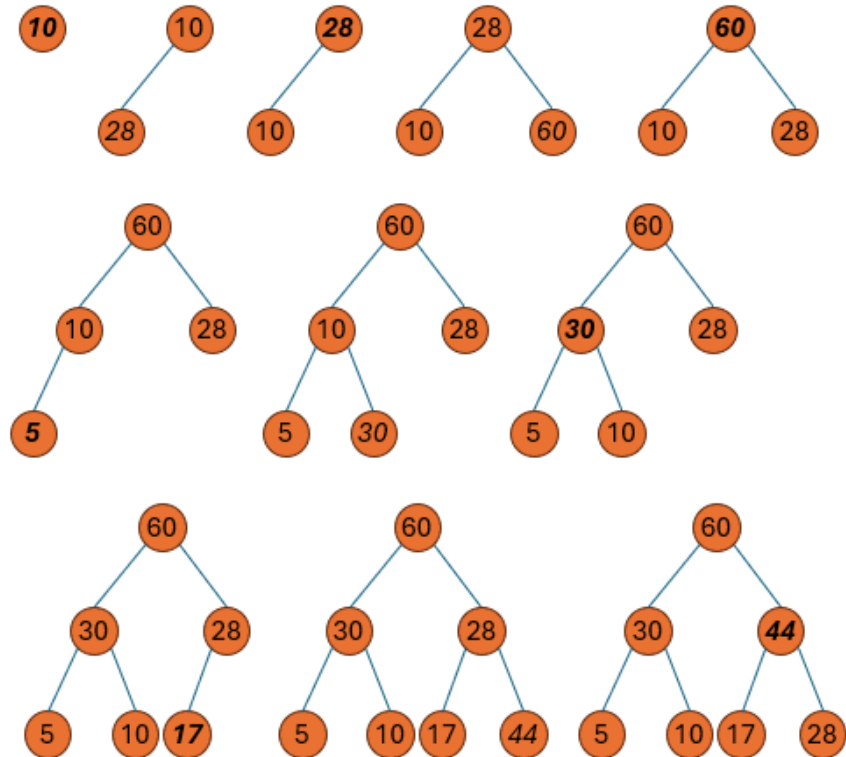
(5) Given the list of values 10, 28, 60, 5, 30, 17, 44, construct a heap using:

(a) the top-down approach (i.e. repeated insertion)

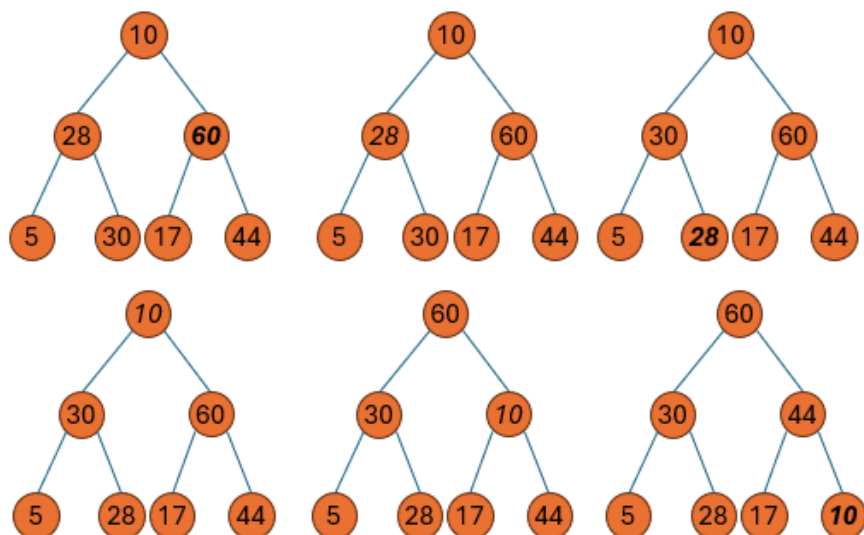
(b) the bottom-up approach

Show all steps in both cases.

(a) Here, the *italicized* node is the new value; **boldface** indicates that it is done moving for now.



(b) Again, *italics* indicates the active node; **boldface** indicates that no more downward moves are needed.



(6) Design and state an efficient algorithm for counting the number of values greater than an input value v in a given heap H containing n elements. State and justify the runtime of your algorithm.

Naïve approach: Just pop the heap repeatedly until v (or something smaller) shows up. This requires $O(n \log n)$ time in the worst case, since we might end up popping every element off the heap. (Note to TAs: This should be worth 7 out of 10 if the explanation and reasoning are good.)

Recursive approach: If the heap is empty or the top of the heap is $\leq v$, return 0. Otherwise, return 1 plus a recursive call on the left subheap plus a recursive call on the right subheap. This will make two recursive calls for each element $> v$, with a constant amount of work (remember that checking the value at the top of the heap is $O(1)$) within each recursive call. Since there can't be more than n elements $> v$, this approach requires $O(n)$ time.